

Security+ 2.3 · SQL Injection

Mestre CyberSec · Lesson 2.3 — SQL Injection (5:09) · [Assistir Vídeo no YouTube](#) · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: Injetar SQL num campo de entrada: 'OR 1=1--' pula o login; DROP TABLE apaga a tabela; UNION SELECT extrai dado. Defesa: PREPARED STATEMENT (parameterized query), input validation, WAF, menor privilégio no banco. Nunca concatenar string pra query.

💡 Key Exam Highlights & Critical Concepts

- SQLi = input injetado no query sem sanitizar
- Defesa nº1: prepared statement / parameterized query
- Defesa nº2: menor privilégio (o banco do user não tem Drop Table)

🎯 Exam Tips & Typical CompTIA Scenarios

- 'input concatenated into a SQL query' = SQL injection
- 'parameterized queries prevent' = defesa do SQLi (prepared statement)

Why is this advanced coverage required for SY0-701? Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• SQL Injection (SQLi)

An injection attack where untrusted input is concatenated into a SQL query, allowing attackers to alter query logic and access or destroy database data.

 **Exam Trap:** Trigger phrase 'input concatenated into a SQL query' = SQLi. Use '--' or ' OR 1=1' as classic textbook payload examples.

• Prepared Statement (Parameterized Query) (N/A)

A coding technique where SQL code and user input are sent to the DBMS in distinct parameters, so input is treated as data only and cannot alter query structure.

 **Exam Trap:** Trigger phrase 'parameterized queries prevent' = the #1 defense for SQL injection on the exam. Prefer this over escaping or WAF when both are options.

• Input Validation (N/A)

Server-side checking of user input against an expected format, type, or range (e.g., allowlist of alphanumeric only) before processing, reducing injection risk.

 **Exam Trap:** Distinguish from sanitization: validation rejects bad input, sanitization cleans it. For SQLi, parameterized query is still preferred even when validation exists.

• Web Application Firewall (WAF)

A layer-7 firewall that inspects HTTP/S traffic to a web application and blocks known malicious payloads such as SQLi, XSS, and command injection patterns.

 **Exam Trap:** WAF is compensating control, NOT a substitute for parameterized queries. On the exam, the best fix for SQLi is always code-level, not network-level.

• Principle of Least Privilege (PoLP)

Granting a database user account only the minimum permissions required (e.g., SELECT/INSERT only), so a successful SQLi cannot DROP tables or escalate.

 **Exam Trap:** Trigger phrase 'database user cannot DROP TABLE' = least privilege mitigated impact. Pair with PoLP for defense-in-depth, not as primary SQLi fix.

• UNION SELECT Attack (N/A)

A SQLi payload using the UNION operator to append a second SELECT statement, exfiltrating data from other tables in the same database through the vulnerable query's output.

 **Exam Trap:** If exam describes 'extracting data via UNION' or 'appended SELECT', it is SQLi, not XSS or command injection. Requires equal column count to succeed.

• DROP TABLE Statement (N/A)

A destructive SQL DDL command often injected via SQLi (e.g., ';' DROP TABLE users--) to delete entire tables and cause denial of service or data loss.

 **Exam Trap:** Trigger phrase 'database user should not have permission to DROP' = least privilege mitigation. Multi-statement execution must be enabled for this payload to work.

• String Concatenation Vulnerability (N/A)

The insecure code pattern of building SQL by joining strings (e.g., 'SELECT * FROM users WHERE name=' + input + '''), which lets attackers break out of the data context into code.

Exam Trap: Any exam question showing query built via '+' or string concatenation is the hallmark vulnerable pattern — fix is parameterized query, not escaping.

✓ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A retail company's login page authenticates users by building this query: `SELECT * FROM Users WHERE username = ' [input] ' AND password = ' [input] '`. A penetration tester enters ' OR 1=1-- in the username field and is logged in as the first user in the table. Which change best addresses the root cause of this vulnerability?

- A) Replace dynamically concatenated SQL with parameterized prepared statements.
- B) Require users to choose longer passwords.
- C) Enable account lockout after three failed attempts.
- D) Enforce TLS 1.3 for all authentication traffic.

QUESTION 2 · EXAM SCENARIO

During a security assessment, a tester finds that a product search page takes the search term and appends it to a query. By submitting a crafted term, the tester causes the database to return usernames and password hashes in the product description column. Which control most directly prevents this type of data extraction?

- A) Deploying a web application firewall in monitor-only mode.
- B) Using parameterized queries for the search operation.
- C) Hiding database error messages from users.
- D) Limiting the search results to ten records per page.

QUESTION 3 · EXAM SCENARIO

A web application connects to its database with an account that has db_owner rights. An attacker exploiting a form input attempts to execute a stacked query that drops the Orders table. Which database-side control would most reduce the impact if the application is successfully injected?

- A) Enabling detailed SQL query logging.
- B) Storing backups on the same database server.
- C) Granting the application account only the permissions required by its functions.
- D) Adding a CAPTCHA to the vulnerable form.

QUESTION 4 · EXAM SCENARIO

A code review finds the following Java snippet: `String q = "SELECT * FROM accounts WHERE acct = '" + acct + "'";` Which replacement best removes the SQL injection flaw?

- A) Replace single quotes in acct with two single quotes before concatenation.
- B) Limit acct to ten alphanumeric characters.
- C) Move the query to a stored procedure that uses EXEC with concatenated acct.
- D) Use a PreparedStatement with a ? placeholder and bind acct as a parameter.

QUESTION 5 · EXAM SCENARIO

A security engineer confirms that a web application firewall is blocking known SQL injection patterns, but the application still concatenates user input into SQL queries. Which additional control most directly reduces the application-layer risk?

- A) Refactor database access to use parameterized queries.
- B) Segment the web server and database server into different VLANs.
- C) Enable file integrity monitoring on the database server.
- D) Require multi-factor authentication for database administrators.

QUESTION 6 · EXAM SCENARIO

A developer adds server-side validation that a customer ID field contains only digits before using it in a database query. Which statement best describes the security value of this control?

- A) It fully prevents SQL injection and removes the need for parameterized queries.
- B) It reduces the attack surface but should be combined with parameterized queries.
- C) It is effective only when client-side JavaScript validation is also enabled.
- D) It replaces the need for least privilege on the database account.

QUESTION 7 · EXAM SCENARIO

A database administrator discovers that the web application's database account has SELECT, INSERT, UPDATE, DELETE, and DROP privileges on all schemas, even though the application only reads from two views and writes to one table. Which action best reduces the potential impact of a compromised application account?

- A) Rename the application database account to a less obvious name.
- B) Enable TLS for all database connections.
- C) Revoke unnecessary privileges and grant access only to required views and table operations.
- D) Move the database to a separate network segment.

QUESTION 8 · EXAM SCENARIO

An application returns raw database errors to users, including table names, column names, and SQL syntax details. An attacker uses these messages to refine SQL injection attempts. Which action best reduces this information disclosure while supporting incident investigation?

- A) Disable database query logging entirely.
- B) Configure the WAF to only log requests without blocking.
- C) Increase the complexity of application user passwords.
- D) Return generic error messages to users and log detailed errors on the server.

QUESTION 9 · EXAM SCENARIO

A legacy application cannot be modified immediately to use parameterized queries. The security architect needs a temporary compensating control at the HTTP layer to block malicious SQL injection requests until the code is fixed. Which control best fits this requirement?

- A) A web application firewall with SQL injection signatures in blocking mode.
- B) Database encryption at rest for all application tables.
- C) Role-based access control for application end users.
- D) Database credential rotation every 30 days.

QUESTION 10 · EXAM SCENARIO

A development team moves several database operations into stored procedures to improve security. A reviewer finds that one procedure builds a query with `EXEC('SELECT * FROM Inventory WHERE Item = ' + @Item + '')`. Which conclusion is most accurate?

- A) Stored procedures always eliminate SQL injection regardless of how they are written.
- B) The procedure remains vulnerable because input is still concatenated into dynamic SQL.
- C) SQL injection only affects ad hoc queries, not stored procedures.
- D) The procedure is safe if the database account has least privilege.

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option A — Replace dynamically concatenated SQL with parameterized prepared statements.

Parameterized prepared statements separate SQL code from data, so the input ' OR 1=1-- is treated as a literal username rather than executable SQL. Longer passwords, lockout, and TLS are useful but do not fix the injection flaw.

Question 2: Option B — Using parameterized queries for the search operation.

The described attack is a UNION SELECT-based SQL injection. Parameterized queries prevent user input from being interpreted as SQL syntax, stopping the injected UNION SELECT. A WAF in monitor mode only observes, error hiding does not stop extraction, and result limits may reduce volume but not the flaw.

Question 3: Option C — Granting the application account only the permissions required by its functions.

Least privilege limits what the injected SQL can do. If the application account cannot drop tables or access unrelated objects, a successful injection causes less damage. Logging, same-server backups, and CAPTCHA do not remove excessive database permissions.

Question 4: Option D — Use a PreparedStatement with a ? placeholder and bind acct as a parameter.

A PreparedStatement with parameter binding treats acct as data and prevents it from changing SQL syntax. Manual quote escaping and length limits are incomplete, and a stored procedure using EXEC with concatenation can remain vulnerable.

Question 5: Option A — Refactor database access to use parameterized queries.

Parameterized queries fix the root cause by preventing input from being executed as SQL. Network segmentation, file integrity monitoring, and MFA for DBAs are useful controls but do not directly eliminate application SQL injection vulnerabilities.

Question 6: Option B — It reduces the attack surface but should be combined with parameterized queries.

Input validation can reject many malicious inputs and reduce attack surface, but it is not a complete SQLi defense by itself. Parameterized queries are still required, and least privilege remains important.

Question 7: Option C — Revoke unnecessary privileges and grant access only to required views and table operations.

Least privilege means granting only the minimum permissions required. Revoking broad rights and limiting access to required objects reduces the impact of SQL injection and other compromises. Renaming, TLS, and segmentation do not address excessive database privileges.

Question 8: Option D — Return generic error messages to users and log detailed errors on the server.

Generic user-facing errors avoid revealing schema and query details, while server-side logging preserves diagnostic information for investigators. Disabling logging harms investigation, WAF logging-only does not reduce disclosure, and password complexity does not address error leakage.

Question 9: Option A — A web application firewall with SQL injection signatures in blocking mode.

A WAF with SQLi signatures in blocking mode can provide temporary detection and prevention at the HTTP layer while the legacy code is fixed. Encryption at rest, RBAC for users, and credential rotation do not inspect or block SQL injection payloads.

Question 10: Option B — The procedure remains vulnerable because input is still concatenated into dynamic SQL.

Stored procedures are not automatically safe. If they construct dynamic SQL with concatenated user input and EXEC, injection is still possible. Least privilege reduces impact but does not fix the vulnerable procedure; injection is not limited to ad hoc queries.